

Laboratorio 8

Humberto Alexander de la Cruz - 23735
Daniel Oswaldo Juárez Herrera - 23709
Dilary Sarahí Cruz López - 231010

Repositorio: <https://github.com/hadelacruz/BERT-multilingue>

Task 2.2

La perplejidad de validación del Mini-GPT fue de **7.12** (rango esperado: 3.5–8.0), con pérdida final de 1.89 en entrenamiento y 1.96 en validación tras 10 épocas. Las cinco muestras de la configuración A son idénticas porque greedy es determinista.

Diversidad medida sobre las 5 muestras de 200 caracteres de cada configuración:

Config.	Caracteres únicos	Trigramas repetidos	Palabras reales
A (greedy)	10.0	89.39 %	100.00 %
B (top- $k=5$, $\tau=0.7$)	29.6	35.15 %	86.61 %
C (top- $k=50$, $\tau=1.0$)	38.2	7.78 %	53.12 %
D (top- $p=0.70$, $\tau=0.7$)	26.2	41.52 %	93.47 %
E (top- $p=0.95$, $\tau=1.0$)	38.4	10.40 %	59.36 %

a. Greedy (configuración A)

Greedy elige $w_t = \arg \max_i P(w_i | w_{<t})$. Es una función determinista del contexto: cuando el modelo vuelve a un contexto casi igual al de antes, produce exactamente la misma salida y entra en un ciclo. En las muestras A el texto colapsa a **The so the so the the the here the the the the the...**: tras un espacio el carácter más probable inicia **the**, y tras **the** vuelve a ser **the**.

La distribución aprendida para el siguiente carácter tiene entropía considerable (pérdida ≈ 1.96 nats, perplejidad ≈ 7 : hay varios caracteres plausibles), pero greedy ignora toda la masa fuera de la moda y siempre toma el patrón más frecuente del corpus. Al no haber aleatoriedad, nada rompe el ciclo. Las métricas lo confirman: A usa 10 caracteres únicos por muestra y el 89 % de sus trigramas están repetidos, frente a 8–10 % en C y E.

b. Configuraciones B y C

Con

$$P(w_i | \text{ctx}) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} :$$

- **Aumentar k de 5 a 50.** Con solo 65 caracteres, $k = 50$ casi no recorta la distribución: quedan candidatos con probabilidad muy baja (letras raras, signos) que pueden ser muestreados, por

lo que aumenta la diversidad. C tiene 38.2 caracteres únicos por muestra frente a 29.6 en B, y 7.8 % de trigramas repetidos frente a 35 %.

- **Aumentar τ de 0.7 a 1.0.** Dividir por $\tau < 1$ amplifica las diferencias entre logits ($z_i/0,7$) y concentra la masa en los primeros tokens (menor entropía); con $\tau = 1$ la distribución queda como la aprendió el modelo, más plana. Ambos cambios apuntan en la misma dirección: más aleatoriedad en C.
- **Coherencia vs. creatividad.** B es la más coherente: 86.6 % de sus palabras existen en el corpus y tiene estructura reconocible (`And that the so strand of here hish here shall`, líneas de diálogo como PUCARCETIO:), aunque abusa de palabras frecuentes (`the`, `and`, `here`). C es la más creativa y ruidosa: mayor variedad, pero solo 53 % de palabras reales y neologismos como `Quet Gancue that shee prorend o nconge of foulht`.

c. Configuración E ($\text{top-}p = 0,95$, $\tau = 1,0$)

El tamaño del conjunto candidato (menor conjunto con masa acumulada $\geq 0,95$) refleja la certeza del modelo: pocos candidatos indica seguridad y muchos indica inseguridad. Lo que sigue se infiere de las muestras; no se registró el tamaño del núcleo en cada paso.

Momentos de alta certeza.

1. El salto de línea tras los dos puntos del nombre de un personaje: `ROMEO: → \n`, `MELICIONGA: → \n` (E2) y `KINGHCHELAS: → \n` (E1).
2. El interior de palabras cuyo prefijo casi determina la continuación, por ejemplo `Thou`, `death` y `hand` en E1, E2 y E5 (tras `dea` o `han` la siguiente letra es casi forzada), y el espacio tras palabras completas como `the` o `and`.

Momentos de incertidumbre.

1. Los nombres de personaje inventados al inicio de línea (`MELICIONGA`, `BUTULOLIUS`, `FRICIRHAF` en E2, E5 y E3), donde puede venir casi cualquier mayúscula.
2. El primer carácter de una palabra poco frecuente tras un espacio o signo de puntuación, donde el modelo divaga: `mugger edvikig hing` (E1), `espeaither dearsiong` (E4).

En resumen, los signos estructurales (`:` $\rightarrow$ `\n`) y la mitad de las palabras son posiciones seguras; los inicios de línea o de palabra y los nombres propios son inciertos.

Task 4.1

a. Máscara causal y Language Modeling

El objetivo de LM factoriza la probabilidad de una secuencia como

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_{<t}),$$

es decir, cada factor predice el siguiente token usando únicamente el pasado. La máscara causal (que suma $-\infty$ a los scores de atención de las posiciones $j > t$ antes del softmax) hace que la representación en t dependa solo de $w_{\leq t}$, de modo que el modelo respeta esa factorización y puede calcular las T predicciones en un solo forward pass sin ver sus propias respuestas.

Con atención bidireccional en el preentrenamiento:

- **El objetivo sería trivial.** Para predecir w_{t+1} la posición t podría atender directamente a w_{t+1} , que es la etiqueta. El modelo aprendería a copiar, la pérdida caería casi a 0 (perplejidad ≈ 1) y no aprendería la estructura del lenguaje. En nuestro Mini-GPT, la pérdida de 1.96 y la perplejidad de 7.12 son significativas precisamente porque el modelo no ve el siguiente carácter.
- **Los factores dejarían de ser probabilidades condicionales válidas.** Condicionar en el futuro rompe la regla de la cadena, así que el modelo ya no definiría una distribución sobre secuencias de la cual muestrear.
- **Habría un desajuste entre entrenamiento e inferencia.** En generación autoregresiva los tokens futuros aún no existen, por lo que el modelo dependería de información que no tendrá al generar.

b. Preentrenar y adaptar: GPT o BERT según la tarea

Evidencia del Task 3. Word2Vec asigna un solo vector por palabra, por lo que la similitud entre distintos usos de *banco*, *copa* y *vela* es exactamente 1.0 en todos los pares. BERT, al ser bidireccional, construye cada vector con el contexto de ambos lados y por eso los diferencia: *banco* (finanzas) vs. (asiento) = 0,58 y vs. (peces) = 0,66; *copa* (trofeo) vs. (vino) = 0,57; *vela* (náutica) vs. (cera) = 0,80. Las similitudes entre palabras distintas son bajas ($\approx 0,32$ – $0,49$) y en el PCA cada palabra forma su propio grupo, con los sentidos separados dentro de él. La separación es parcial (*vela* conserva 0.80), pero es una capacidad que un embedding estático no tiene.

Por qué GPT no lo hace con la misma facilidad. Por la máscara causal, la representación de un token solo conoce su contexto izquierdo: en “el banco de peces” el token *banco* se codifica sin saber que después viene *peces*. A cambio, esa misma restricción es la que le permite generar texto, algo que BERT no puede hacer de forma autoregresiva.

Solución del paradigma. Ambos modelos aprenden una representación general del lenguaje con un objetivo auto-supervisado sobre grandes corpus (masked LM en BERT, predicción del siguiente token en GPT) y luego se adaptan a la tarea con poco costo (fine-tuning con una cabeza nueva, o prompting). Así, la tensión entre ambos deja de ser un problema: solo se elige la arquitectura según la tarea, sin entrenar ninguna desde cero.

Tarea	Modelo natural
Clasificación, NER, similitud, desambiguación, búsqueda	BERT: necesita contexto completo, como en el Task 3
Generación, completado, diálogo	GPT: necesita $P(w_t \mid w_{<t})$, como en el Task 2